

**SENTRYX – AN AI-BASED VEHICLE AUTHORIZATION SYSTEM
USING AUTOMATIC LICENSE PLATE RECOGNITION BUILT WITH
YOLOV8N AND PADDLEOCR**



**MUHAMMAD SAAD
SHAHID MANZOOR
RANA ASHHAD**

STUDENTS OF NATIONAL UNIVERSITY OF TECHNOLOGY (NUTECH)

**SUPERVISOR
DR. INAYATULLAH KHAN**

**NATIONAL ENGINEERING AND SCIENTIFIC COMMISSION
(NESCOM)**

Table of Contents

Abstract	ii
1. Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	1
1.4 Report Structure	2
2. Literature Review	2
2.1 Traditional Vehicle Access Control Methods	2
2.2 Deep Learning in Object Detection	2
2.3 Optical Character Recognition Approaches	3
2.4 Existing ALPR Frameworks and Gaps	3
3. Methodology	4
3.1 System Architecture Overview	4
3.2 Data Acquisition and Preprocessing	4
3.3 Detection Model Selection and Training	4
3.4 Recognition and Normalization Pipeline	5
3.5 Authorization and Access-Control Logic	5
4. Implementation	6
4.1 Technical Environment	6
4.2 Core Pipeline Implementation	7
4.3 Multi-Source Input Handling	7
4.4 Web Application and Database Integration	8
5. AI-Driven Software Engineering: Paradigms, SDLC Integration, and Practical Application	10
5.1 AI in Modern Software Engineering	10
5.2 AI Across the Full-Stack Development Life Cycle	11
5.3 Case Study: AI-Assisted Engineering in SENTRYX	11
5.4 Risks, Security, and Code Quality	12
5.5 Future Trajectory	13
6. Experimental Results and Discussion	13
6.1 Test Methodology	13
6.2 Detection and Recognition Performance	14
6.3 System Level Results	15
6.4 Limitations	16
7. Conclusion and Future Work	17
7.1 Summary of Contributions	17
7.2 Future Enhancements	17
References	18

ABSTRACT

Conventional checkpoint verification relies on manual logging, introducing operational delays, shift inconsistencies, and untracked entry records. To resolve these limitations, this report presents SENTRYX, an AI-based vehicle authorization framework utilizing Automatic License Plate Recognition (ALPR). The system captures vehicle license plates from image, video, or live webcam streams using a custom-trained YOLOv8n detector. Recognized text is extracted via a recognition-only PaddleOCR configuration optimized to process both single-line and two-line Pakistani plate formats. Extracted strings are normalized into a unified query format and cross-referenced against a PostgreSQL database to output a logged access decision, all executing on edge-level CPU hardware without requiring a dedicated GPU.

The detection model achieved a precision of 0.979 and recall of 0.969 (mAP@50 of 0.991) on a validation set of 1,765 images, with inference at 1.9 ms per image. Component-level testing of the recognition stage showed 100% of test crops produced a non-empty, high-confidence read, with an exact ground-truth match rate of 57.1% on isolated single-frame crops, a figure the system is explicitly designed to work around through multi-frame consensus and confidence-based bypass logic during video and webcam operation, rather than relying on any single frame. In one recorded video-mode session, the system tracked 11 unique vehicles across 1,468 frames and correctly read 11 of 12 detected plates (91.7%).

Beyond the recognition pipeline, the system is deployed behind a complete web application: a public portal for vehicle registration and status lookup, and a protected administrator portal for reviewing registration requests, managing authorized vehicles, and auditing access history, backed by a three-table Neon PostgreSQL schema and a FastAPI service layer. Thirteen scripted functional tests covering this application layer, including its authentication and validation boundaries, all passed. The report also documents the role of agentic AI coding tools in the system's development, situating this practical use within a broader review of AI-assisted software engineering research. The resulting system demonstrates a complete, validated path from a raw camera frame to a logged, database-backed authorization decision, and is presented as a working software foundation ready for further deployment hardening.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Controlling vehicle access to a facility is a routine security requirement, yet most institutions still rely on manual verification via guards or paper registers. While this works at a small scale, it breaks down as traffic grows—causing entry delays, unsearchable records, and inconsistent decisions based on human judgment. Automatic License Plate Recognition (ALPR) solves this by using a camera to capture a plate, a vision system to detect and read the text, and a database to verify authorization, automating access without changing physical plates.

SENTRYX applies this ALPR premise to build an automated vehicle authorization system tailored for controlled environments like universities or restricted facilities. By combining object detection, optical character recognition, and database-backed authorization into a single pipeline, SENTRYX generates instant access decisions directly from camera frames with zero manual intervention.

1.2 Problem Statement

Manual, guard-based vehicle verification at access points is slow, inconsistent across shifts and operators, and produces no reliable digital record of who entered or when. There is a need for a system that can take a live or uploaded image of a vehicle, correctly identify its license plate under real-world conditions (varying lighting, angle, and plate formats), determine whether that vehicle is authorized to enter, and log the decision without requiring a person to read or type the plate manually.

1.3 Research Objectives

The project set out to:

- Detect license plates in vehicle images using a trained object detection model.
- Recognize the alphanumeric text on a detected plate under varied real-world image conditions, including two-line plate formats common on Pakistani vehicles.
- Normalize recognized text into a consistent format to match against stored records.
- Cross-reference the recognized plate against an authorization database and return an access decision.
- Log every access attempt, authorized or not, for later review.
- Package the system behind a web interface so authorization records can be managed without direct database access.

1.4 Report Structure

Chapter 2 reviews existing work relevant to vehicle access control, object detection, and OCR. Chapter 3 describes the system's methodology and architecture. Chapter 4 details the implementation. Chapter 5 examines the role AI-assisted software engineering tools played in building the system, situated within broader research on AI in the software development life cycle. Chapter 6 presents experimental results. Chapter 7 concludes the report and outlines future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Traditional Vehicle Access Control Methods

Before automated recognition, vehicle access control relied on two main approaches: manual verification by security personnel and RFID-based tag systems. RFID systems place a passive or active tag on the vehicle windshield; a reader at the gate detects the tag and checks it against a permit database before releasing the barrier [1], [2]. These systems are fast and work in poor weather since they don't depend on a camera reading a physical plate [2], but they require every vehicle to carry a compatible tag, which makes them unsuitable for visitor traffic, unregistered vehicles, or any facility that can't guarantee tag compliance across all users. Manual guard-based checks avoid this dependency but scale poorly and leave no reliable record beyond a paper register.

Camera-based License Plate Recognition (LPR) has since been positioned as a layer that removes the tag requirement entirely: the camera reads whatever plate is already on the vehicle, and the gate opens on a database match [3]. Commercial deployments increasingly combine both: RFID for high-frequency permit holders and LPR for general traffic in the same lane [3], which reflects a broader industry conclusion that camera-based recognition, not tag hardware, is the more general solution when the vehicle population isn't fully known in advance. This is the gap SENTRYX is built to address.

2.2 Deep Learning in Object Detection

Plate detection is the first stage of any LPR pipeline, and its accuracy sets a ceiling on everything downstream. Early ALPR systems relied on handcrafted edge and color features to locate a plate region [4], an approach that degraded quickly under real-world lighting and angle variation. The shift to deep learning-based detectors addressed this by learning plate features directly from data rather than encoding them by hand [5].

Among these detectors, the YOLO (You Only Look Once) family has become the dominant choice for ALPR specifically because it treats detection as a single regression pass over the image rather than a multi-stage region-proposal process, which keeps inference fast enough for near-real-time use [6]. YOLOv8, released by Ultralytics in 2023, replaced the anchor-based detection head used in earlier YOLO versions with an anchor-free design and a decoupled classification/localization head, improving both accuracy and inference speed across model sizes [7], [8]. The Nano variant (YOLOv8n) trades some accuracy for a small parameter count, making it viable on CPU-only hardware; the constraint this project operates under. Multiple recent ALPR studies confirm YOLOv8 as a practical detection backbone for plate localization specifically, reporting detection precision above 99% on curated regional datasets [9], though these figures come from clean benchmark conditions rather than field deployment.

2.3 Optical Character Recognition Approaches

Once a plate region is detected and cropped, OCR converts the pixel region into text. General-purpose OCR engines commonly used in ALPR research include Tesseract, EasyOCR, and PaddleOCR, each built on a different underlying architecture. Tesseract remains widely used for

its language flexibility and integration simplicity [10]. EasyOCR, based on a CRNN (Convolutional Recurrent Neural Network) architecture, has been paired with YOLOv8 in several ALPR studies as a lightweight, dependency-light recognition stage [6].

PaddleOCR takes a different structural approach: rather than a single end-to-end recognition network, it separates the problem into a text-detection stage (locating individual text lines within the cropped region, using a differentiable binarization detector) and a text-recognition stage that reads each located line [11], [12]. This separation matters for plates with more than one line of text, a layout common on Pakistani plates since a single-pass recognizer tuned for one line of Latin text tends to misread or concatenate a two-line plate incorrectly. Baidu's published benchmarks for the PP-OCR model series report consistent accuracy gains across successive versions while keeping the model small enough for CPU deployment [12], which is the basis for this project's use of PaddleOCR's recognition-only mode over a general-purpose single-line OCR engine. Comparative studies of OCR engines on plate text specifically report that no single engine generalizes cleanly across all script types and plate layouts [10], reinforcing that engine choice needs to be matched to the specific plate format a system will encounter rather than assumed from general OCR benchmarks.

2.4 Existing ALPR Frameworks and Gaps

Published ALPR systems typically report strong accuracy on their own benchmark datasets, precision and recall figures in the high 90s are common in the literature [9], [13] but these numbers are consistently qualified by dataset composition. Systems trained and tested on plates from a single country's format, under a narrow range of weather and lighting conditions, do not reliably transfer that accuracy to a different plate format or an unconstrained deployment environment. Pakistani plates specifically present a layout challenge that most published benchmark datasets don't cover: a two-line format combining a region code, a short alphanumeric identifier, and a numeric segment, which existing single-line-optimized ALPR pipelines are not designed to parse correctly.

A second, less-discussed gap in the published literature is architectural rather than algorithmic: most academic ALPR papers evaluate detection and recognition accuracy in isolation, with far fewer describing how the recognized plate connects to an actual authorization decision, a persistent access record, or a usable interface for the people managing that access list. Detection and recognition are necessary but not sufficient for a deployable access-control system, the recognized text still has to be normalized into a lookup-safe format, checked against a live authorization store, and logged in a way that supports later review. SENTRYX is positioned specifically to close this second gap: it treats detection and recognition as the front end of a complete pipeline rather than as the end product, and is built around a plate format that the existing published benchmarks largely don't address.

CHAPTER 3: METHODOLOGY

3.1 System Architecture Overview

SENTRYX is organized as a pipeline with a clear separation of responsibility between four stages: detection, recognition, authorization, and presentation. An input frame from an uploaded image, a video file, or a live webcam is passed to a YOLOv8n detector that locates the license plate region. The cropped plate is then handed to a PaddleOCR recognition stage, which returns candidate text. A normalization step converts that text into a consistent lookup format, which is checked against an authorization store; the result and a log entry are then made available to the user-facing layer, as shown in Fig 3.1.

This separation matters for two reasons. First, it means the vision engine (detection, recognition, tracking) can be developed, tested, and corrected independently of how its output is eventually displayed or stored. Second, it keeps the authorization decision itself simple and auditable: the vision pipeline never decides who is authorized, it only produces a plate string, and the decision is made by a single lookup against a record store.



Fig. 3.1 System Architecture Diagram.

3.2 Data Acquisition and Preprocessing

Detection training data was sourced through Roboflow datasets, using a custom-annotated dataset of vehicle images with license plate bounding boxes, built specifically around Pakistani plate formats rather than relying solely on international benchmark datasets, which are dominated by single-line plate layouts. Each image was annotated with a single object class (`license_plate`) in YOLO format.

Before detection, no plate-specific preprocessing is applied to the full frame, the detector operates on the raw input image. Preprocessing is instead applied after detection, on the cropped plate region, where enhancement variants (contrast adjustment, sharpening, grayscale conversion) are generated to give the recognition stage multiple candidate views of the same plate rather than relying on a single crop.

3.3 Detection Model Selection and Training

YOLOv8n was selected as the detection backbone specifically for its balance between accuracy and CPU inference speed, since the target deployment hardware does not assume GPU availability. The model was trained on the Roboflow-sourced dataset [14] using Google Colab, starting from Ultralytics' pretrained YOLOv8n weights rather than training from scratch, which reduces the amount of labeled data needed to reach usable accuracy on a narrow, single-class detection task (Fig. 3.2).

```

EarlyStopping: Training stopped early as no improvement observed in last 15 epochs. Best results observed at epoch 73, best model saved as best.pt.
To update EarlyStopping(patience=15) pass a new patience value, i.e. 'patience=300' or use 'patience=0' to disable EarlyStopping.

88 epochs completed in 2.584 hours.
Optimizer stripped from /content/runs/detect/rbflw_yolov8_trainedmodel/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/rbflw_yolov8_trainedmodel/weights/best.pt, 6.2MB

Validating /content/runs/detect/rbflw_yolov8_trainedmodel/weights/best.pt...
Ultralytics 8.4.129 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,085,843 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95	100%	56/56	3.8it/s	14.6s
all	1765	1840	0.979	0.969	0.991	0.706				

```

Speed: 0.2ms preprocess, 1.9ms inference, 0.0ms loss, 1.9ms postprocess per image
Results saved to /content/runs/detect/rbflw_yolov8_trainedmodel

```

Fig. 3.2 Trained YOLOv8n model results in Google Colab.

Training was run for 100 epochs with a patience-based early-stopping criterion of 15 epochs, a batch size of 16, and an input resolution of 640×640. The resulting weights were validated on a held-out split before being adopted into the pipeline.

3.4 Recognition and Normalization Pipeline

Detected plate crops are passed through ByteTrack [15] across video and webcam frames so that a plate already being tracked does not require OCR to be re-run on every single frame. Recognition is triggered on a tracked object rather than on every incoming frame, which reduces redundant computation on a CPU-only system.

Recognition itself uses PaddleOCR's recognition-only mode (PP-OCRv6, TextRecognition class) rather than PaddleOCR's full detection + recognition pipeline, since the plate region has already been localized by YOLOv8n. Pakistani plates commonly use a two-line layout, and an aspect-ratio threshold intended to distinguish single-line from two-line plates was found to be unreliable across the range of crop qualities produced by the detector. The pipeline instead computes both a whole-crop recognition attempt and a split (top-line/bottom-line) recognition attempt for every plate, and a scoring function selects the higher-confidence result rather than committing to one layout assumption ahead of time.

Recognized text is then normalized: converted to uppercase, stripped of non-alphanumeric noise, and assembled into a single canonical string that matches the format used in the authorization store. Region-label tokens (e.g., "ISLAMABAD, PUNJAB, ICT-ISLAMABAD, KHYBER PAKHTUNKHUA") are filtered using substring matching rather than exact matching, since OCR frequently merges a region label into an adjacent token during recognition.

3.5 Authorization and Access-Control Logic

The normalized plate string is checked against an authorization table, and the result (authorized or unauthorized) along with a timestamp is written to an access log. This authorization and logging layer, along with the web interface used to manage it, is implemented using Neon PostgreSQL and FastAPI. The vision engine communicates with this layer through a defined interface rather than accessing the database directly, keeping the recognition logic independent of how records are stored or presented. Role-based access to this layer distinguishes an administrator, who can view and manage both the authorization table and the access log, from a standard user, who can register a vehicle and check only that vehicle's own status.

CHAPTER 4: IMPLEMENTATION

4.1 Technical Environment

The technical stack and system configuration for the SENTRYX system are structured into specialized layers for vision processing, backend services, and cloud storage. Table 4.1 details the hardware assumptions, programming environments, and software frameworks utilized across development and deployment.

TABLE 4.1

TECHNICAL STACK & DEVELOPMENT ENVIRONMENT SPECIFICATIONS

Category	Technology/Tool	Role & Deployment Scope
Inference Hardware	CPU-only System	Execution environment for the vision engine at the access point.
Training Hardware	Google Colab (NVIDIA T4 GPU)	Accelerated training phase for the custom YOLOv8n detector.
Language & Environment	Python 3.12.10 (paddleocr_env)	Isolated virtual environment for vision components and logic.
Core AI/Vision Engines	YOLOv8n & PaddleOCR	Vehicle license plate detection (YOLOv8n) and optical character recognition (PaddleOCR).
Development IDE & Tools	VS Code & Claude Code	Primary IDE and CLI assistant for retraining, integration, and queuing implementation.
Backend Framework	FastAPI	RESTful web application API layer and request routing.
Frontend Stack	React + Vite	User interface build tools and dynamic web interface frontend.
Database Layer	Neon PostgreSQL	Cloud-managed relational database for system logging and access control persistence

4.2 Core Pipeline Implementation

The vision engine shown in Fig 4.1 is implemented as a sequence of discrete, independently testable functions rather than a single monolithic script, which keeps each stage of the pipeline replaceable without requiring changes to the stages around it. The detection stage loads the trained YOLOv8n weights once at startup and runs inference on each incoming frame, returning a bounding box and confidence score for any detected plate region. The cropped region is passed to the recognition stage, where PaddleOCR's recognition-only mode is applied to both a whole-crop candidate and a split-line candidate, and a scoring function selects the higher-confidence result. The selected text is normalized into a canonical plate string and passed to the authorization interface described in Section 4.4.

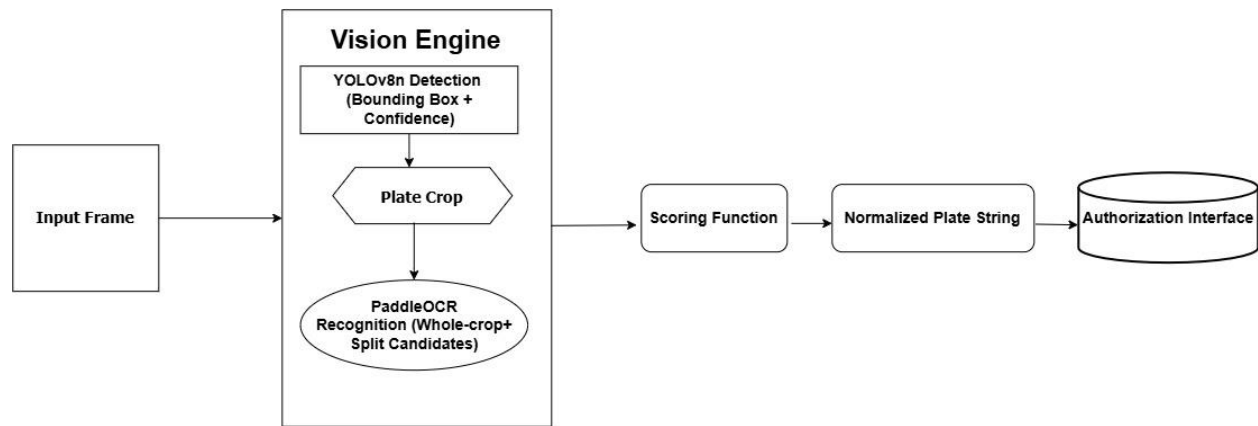


Fig. 4.1 Core Pipeline Data Flow.

4.3 Multi-Source Input Handling

The pipeline accepts three input types through a shared interface: static images, video files, and a live webcam feed. For video and webcam input, ByteTrack is used to maintain a persistent identity for each detected plate across frames so that a plate already being tracked is not re-submitted to the recognition stage on every single frame and the recognition is triggered at a lower tracking-informed frequency instead.

To handle these input types without one blocking another particularly under the CPU constraints of the development hardware, a queuing system was implemented to manage frame throughput between the detection and recognition stages. This decouples frame capture from recognition timing, so a slower OCR pass does not stall the incoming video or webcam stream. Performance testing across two machines (a 2-core development laptop and a 6-core university workstation) confirmed that throughput scales with available CPU cores, consistent with the queuing design rather than indicating a bottleneck in the pipeline logic itself, as mapped in Fig 4.2.

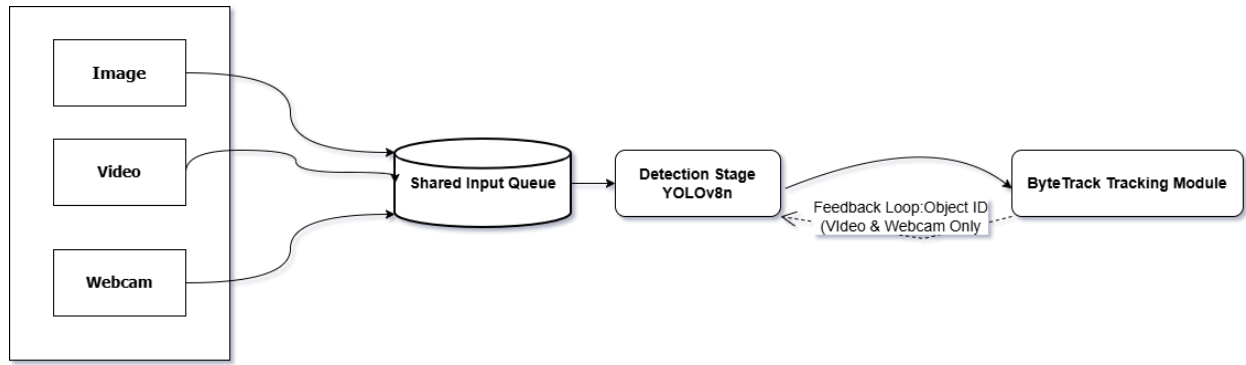


Fig. 4.2 Multi-Source Input Queuing Diagram.

4.4 Web Application and Database Integration

The vision engine exposes its output through a defined interface consumed by the FastAPI backend, keeping the recognition logic independent of how records are stored or displayed. The backend separates public-facing and administrative functionality behind distinct endpoint groups, backed by a Neon PostgreSQL database with three core tables, as schematized in Fig. 4.5: `authorized_vehicles`, the master record of vehicles permitted access; `vehicle_registration_requests`, holding public registration submissions pending review; and `access_logs`, recording every verification attempt with its outcome, confidence, and timestamp.

The React frontend is split into two separate interfaces. A public User Portal lets any visitor submit a vehicle registration request with an owner name, vehicle model, and photograph, and check that request's status (not registered, pending, approved, or rejected) without needing an account. A protected Administrator Portal, reached through a separate login gate, provides the operational side of the system: a live dashboard, the verification console (Fig. 4.3), a registration-request review queue where submissions are approved or rejected (Fig. 4.4), full vehicle-record management, and the access-log history. A registration request only becomes an authorized vehicle once an administrator approves it; public submissions never write directly into the authorization table, as summarized in Table 4.2.

TABLE 4.2

BACKEND API ENDPOINTS SPECIFICATIONS

Endpoint	Method	Purpose	Access
/api/public/register-vehicle	POST	Create a public vehicle-registration request with image	Public
/api/public/vehicle-status/{plate_number}	GET	Check registration state of a plate	Public
/api/admin/login	POST	Validate admin credentials and issue session token	Public

/api/admin/logout	POST	Invalidate administrator session	Authenticated session
/api/admin/vehicle-requests	GET	List registration requests for review	Admin
/api/admin/vehicle-requests/{id}/approve	PUT	Approve a pending request and create authorized vehicle	Admin
/api/admin/vehicle-requests/{id}/reject	PUT	Reject a pending request and optionally record notes	Admin
/api/verify	POST	Accept a vehicle image and return structured verification result	Verification Service
/api/dashboard	GET	Return live dashboard statistics	Application Data
/api/vehicles	GET / POST	List or register managed vehicles	Admin
/api/vehicles/{plate_number}	PUT / DELETE	Edit or remove a managed vehicle	Admin
/api/logs	GET	Return chronological access logs	Application Data

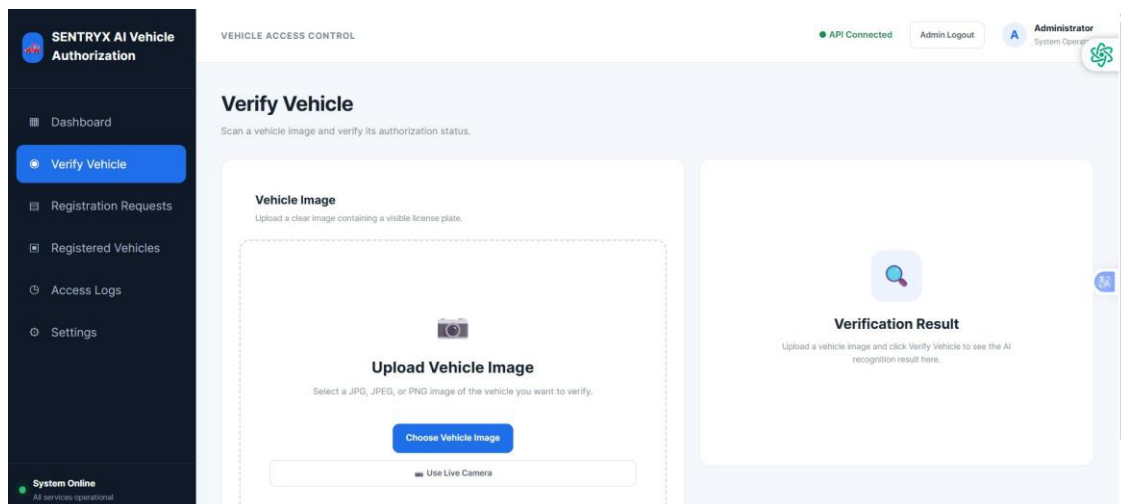


Fig. 4.3 Vehicle Verification Console in Administration Portal.

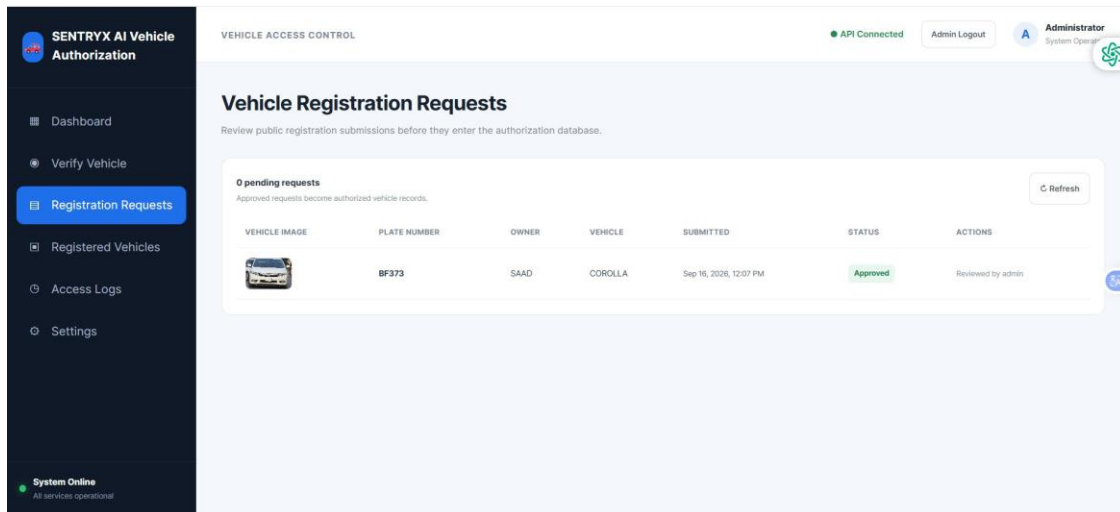


Fig. 4.4 Registration Requests Review Queue in Administration Portal.

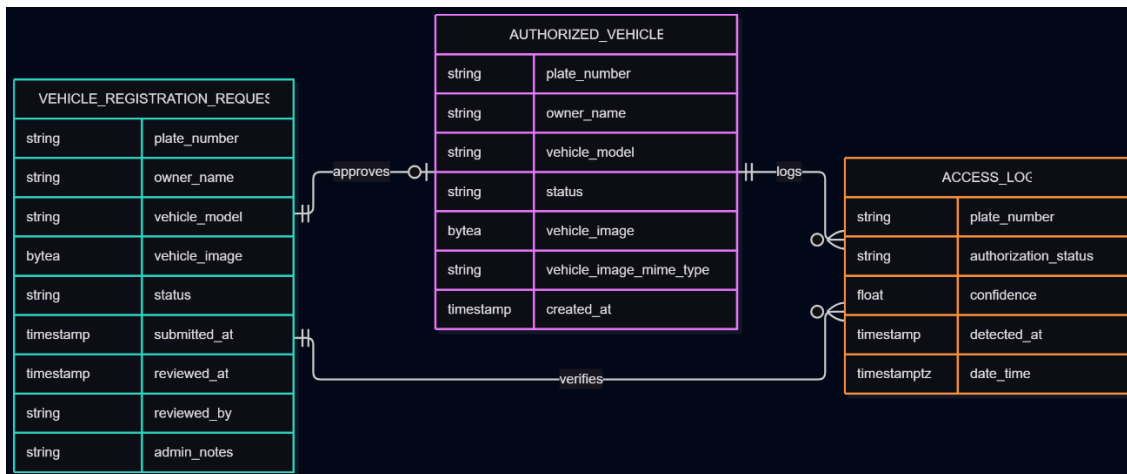


Fig. 4.5 Database Schema Integrated with Web Application

CHAPTER 5: AI-DRIVEN SOFTWARE ENGINEERING: PARADIGMS, SDLC INTEGRATION, AND PRACTICAL APPLICATION

5.1 AI in Modern Software Engineering

Software development has traditionally depended on manual execution at every stage; writing code, tracing bugs, and refactoring structure by hand. That has started to change with two distinct generations of AI tooling. The first generation, exemplified by GitHub Copilot [16], works at the level of inline suggestion: it predicts the next line or function based on immediate context, functioning as an extension of the editor rather than an independent participant in development [17]. Studies of Copilot's effect on developer output report real but variable gains, one controlled trial found developers completed a standard task 55.8% faster with Copilot [17], while a separate trial on more complex, real-world code found the improvement closer to 21%, suggesting the benefit shrinks as task complexity grows.

The second, more recent generation is agentic: tools such as Claude Code operate above the level of single-line suggestion, reading a codebase, planning a multi-step change, executing it, and verifying the result through the terminal rather than waiting for line-by-line approval [18]. Anthropic describes Claude Code as a terminal-resident agent that understands a project's structure and can carry out coordinated, multi-file changes rather than isolated edits [18]. This distinction is the basis for how the tool was used in SENTRYX's own development, described in Section 5.3.

5.2 AI Across the Full-Stack Development Life Cycle

Beyond code generation, AI tooling has extended into two other stages of the development life cycle relevant to a project like this one: refactoring and testing. Refactoring a script-based prototype into an API-exposed service is a common pattern in AI-assisted engineering, tools built for this purpose typically trace dependencies across a monolith's files [19], identify natural extraction boundaries, and scaffold the surrounding interface code (request handling, validation, routing) that a manual refactor would otherwise take substantial developer time to produce [20]. This mirrors the transition SENTRYX's own pipeline underwent, from an initial script-based prototype to the modular pipeline described in Chapter 4.

On testing, LLM-based test generation has been shown to reach meaningful coverage without manual test-case curation. One widely cited study reported a median statement coverage of 70.2% [20] on unseen code using an LLM-based tool, notably higher than a prior non-LLM baseline. A separate analysis of AI-authored commits in real repositories found these tests are not merely superficially plausible: they show measurably different structural patterns (more assertions, more linear control flow) while reaching coverage comparable to human-written tests [21]. The same body of work is consistent on one point, though: AI-generated output, testing included, still requires human review before being trusted in a system where a false result has real consequences, i.e., the principle directly relevant to an authorization-granting system like SENTRYX.

5.3 Case Study: AI-Assisted Engineering in SENTRYX

Claude Code, run inside VS Code, was used across three parts of the vision engine's development: adapting the pipeline to a newly retrained YOLOv8n model, migrating recognition from an earlier OCR engine to PaddleOCR, and building the queuing system that lets the pipeline handle image, video, and webcam input without one blocking another (Section 4.3). In each case, the tool was used interactively, reasoning through a specific, scoped problem in the terminal, applying a change, and verifying it, rather than generating the pipeline from scratch, as detailed in Table 5.1. `git commit` was used deliberately after each verified change, functioning as a manual checkpoint and review gate rather than letting the agent commit unsupervised; the `--continue` flag was used to resume a session's context across a multi-step task rather than re-establishing it each time.

TABLE 5.1

TRADITIONAL VS. AI-ASSISTED DEVELOPMENT WORKFLOW

Development Task	Traditional (Manual) Approach	AI-Assisted Approach (Claude Code)
Adapting the pipeline to a newly retrained YOLOv8n model	Manual review of the new model's output format, followed by manual adjustment of every downstream function that consumes it	Agent-assisted tracing of the model's output structure and coordinated adjustment of dependent pipeline code
Migrating recognition from an earlier OCR engine to PaddleOCR	Manual rewrite of the recognition call and its output-parsing logic, tested incrementally by hand	Agent-assisted refactor of the recognition interface, checkpointed with <code>git commit</code> after each verified stage
Diagnosing the two-line plate recognition failure	Manual trial-and-error adjustment of aspect-ratio thresholds across sample plates	Agent-assisted reasoning toward scoring both whole-crop and split candidates rather than gating on a threshold
Building the image/video/webcam queuing system	Manual design and hand-written concurrency/buffering logic, debugged through direct observation	Agent-assisted implementation, with throughput behavior verified afterward across two hardware configurations
Maintaining context across multi-step development sessions	Re-establishing context manually after each interruption	<code>--continue</code> flag used to resume a session without re-explaining prior state

This pattern reflects the general distinction drawn in Section 5.1: the tool operated on well-scoped, verifiable subtasks adapting an interface to a new model's output format, restructuring a recognition call, decoupling two stages of a pipeline, with the developer retaining review authority

at each checkpoint. The database and web-application layer (Chapter 4) followed a comparable review-gated pattern within its own workflow, using Claude and ChatGPT for scaffolding and debugging rather than Claude Code specifically, though this falls outside the scope of the present case study.

5.4 Risks, Security, and Code Quality

Three risk categories are directly relevant to a system like SENTRYX. The first is package hallucination: LLMs occasionally reference software dependencies that don't exist, and a 2025 study found this happens in roughly 20% of generated package references across 16 code-generation models, with more than half of hallucinated names recurring consistently enough to be pre-registered by an attacker as a real, malicious package [22]. The practical safeguard is straightforward; verify any AI-suggested dependency before installing it, and pin dependencies through a lockfile rather than trusting a suggestion at face value.

The second is code-level security, specifically around database access. Audits of AI-generated code consistently find injection vulnerabilities among the most common defect categories, with one widely cited study reporting exploitable vulnerabilities in around 40% of Copilot-generated samples [23]. For a system whose authorization decision depends on a live database query, an unreviewed, string-concatenated query would be a critical vulnerability rather than a cosmetic bug. The standard mitigation, parameterized queries with mandatory human review of any AI-generated data-access code applies directly here.

The third is specific to computer vision: detection and recognition accuracy trained on curated data does not automatically hold up under real-world conditions. Controlled studies simulating weather distortion found some conditions reduced ALPR accuracy to near zero, and that recognition accuracy drops sharply once plate tilt passes roughly 70 degrees [24]. This is directly relevant to SENTRYX's move from static test images to live webcam and video input (Section 4.3), and is the reasoning behind including multiple enhanced image candidates at the recognition stage (Section 3.4) rather than relying on a single, unenhanced crop.

5.5 Future Trajectory

Two research directions highlight the future trajectory of tools like Claude Code and their relevance to systems like SENTRYX. The first is agent-native testing, where agents continuously align tests with evolving codebases and adapt to interface changes rather than treating them as bugs [25]. The second is broader autonomous operation, as over half of enterprises now run AI agents in production, shifting the focus toward how much unsupervised authority these agents should be granted [26].

However, increased autonomy requires strict scoping because unmonitored agents can make rapid mistakes. For access-control systems where errors impact physical security, full autonomy is impractical. The ideal model remains the one used in SENTRYX: assigning well-defined, mechanical tasks to the AI agent while reserving human review and final approval for all critical authorization logic.

CHAPTER 6: EXPERIMENTAL RESULTS AND DISCUSSION

6.1 Test Methodology

Detection and recognition were benchmarked using 8 representative test images across single- and two-line Pakistani plate formats, supported by a 4-image confirmatory batch and extended video-mode multi-frame testing. Additionally, normalization was evaluated across 15 edge cases for region-label removal and character correction, authorization logic was tested against 14 registry lookups, and access logging was verified through atomic append tests with automatic log restoration.

6.2 Detection and Recognition Performance

The custom-trained YOLOv8n detector was evaluated on its validation split after 88 training epochs (early-stopped from a planned 100, with the best-performing weights retained from epoch 73) as summarized in Table 6.1, and Table 6.2 & Table 6.3 for both image and video batch detection and recognition performances. Fig 6.1 shows a representative detection output on a test image.

TABLE 6.1

YOLOV8N DETECTION PERFORMANCE

Metric	Value
Validation Images	1,765
Annotated plate instances	1,840
Precision	0.979
Recall	0.969
mAP@50	0.991
mAP@50–95	0.706
Inference speed	1.9 ms/image
Training time	2.5 hours (Tesla T4 GPU)

TABLE 6.2

PADDLEOCR RECOGNITION PERFORMANCE (8-IMAGE PRIMARY BENCHMARK)

Metric	Value
Tested Images	8 (single-line and 2 two-line formats)
Non-empty, high-confidence reads	8/8 (100%)
Exact ground-truth match rate	4/7 (57.1%)
Average OCR Confidence	≥ 0.88
Recognition Latency	$\sim 4.0 - 4.1$ s/crop (CPU)

Representative end-to-end traces illustrate this behavior concretely. For MN1367, detection confidence was 0.812, and the original (unsplit) crop was read at 1.00 OCR confidence with an exact match on the first attempt. By contrast, AKF938 detected at 0.822 confidence was read as MAKF938, a leading-character artifact that caused a mismatch despite a high 0.96 OCR confidence, illustrating that strong recognition confidence does not by itself guarantee a correct read on an isolated frame.



Fig. 6.1 YOLOv8n Detection Observation on a Pakistani license plate

TABLE 6.3

VIDEO-MODE RECOGNITION PERFORMANCE

Metric	Value
Total frames in video	1,468
Frames processed (sampled)	146
Unique vehicles tracked	11
Vehicle detections processed	97
License plates detected	12
OCR successful	11/12 (91.7%)
OCR calls made	12
Vehicles finalized and logged	8
Authorization outcome	0 authorized, 8 unauthorized

6.3 System Level Results

The web application layer was evaluated through 13 scripted test cases covering the public registration path, the administrator review workflow, and core security/validation behavior, detailed in Table 6.4.

TABLE 6.4
WEB APPLICATION FUNCTIONAL TEST RESULTS

Test Case	Expected Result	Status
Public portal loads	User sees registration and status services	Pass
User registration request	Form + image creates pending request	Pass
Vehicle status lookup	Registered/pending/rejected/not-registered state returned	Pass
Administrator login	Valid credentials unlock protected application	Pass
Registration request list	Administrator sees current requests and metadata	Pass
Approve request	Approved request becomes an authorized vehicle	Pass
Reject request	Request becomes rejected, notes optionally stored	Pass
Dashboard data	Counts reflect persisted database state	Pass
Vehicle verification	Upload/camera request returns structured result	Pass
Access logging	Attempt stored with decision, confidence, and time	Pass
Protected management request	Invalid token rejected	Pass
Vehicle image limit	Oversized upload rejected	Pass

6.4 Limitations

The system's current limitations follow from constraints inherent to its design and testing scope:

- Detection accuracy is validated against the training dataset's own distribution; performance on plate conditions underrepresented in that data (severe damage, non-standard formats, extreme viewing angles) is not separately confirmed.
- The gap between mAP@50 and mAP@50–95 (Section 6.2) indicates bounding-box precision loosens under stricter overlap criteria, which can affect plate-crop quality and, in turn, recognition accuracy on marginal detections.
- CPU-only deployment limits real-time throughput on lower-core hardware, as observed during development (Chapter 4).
- Registration-request review currently depends on an administrator actively checking the queue; there is no automated notification when a new request arrives.
- Multi-frame OCR consensus (Section 3.4) was not meaningfully exercised during video testing, since detected plates consistently cleared the high-confidence bypass threshold and were finalized from single reads.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Summary of Contributions

SENTRYX demonstrates a complete, working path from a license-plate image to a logged authorization decision. A custom-trained YOLOv8n detector localizes the plate region, while PaddleOCR extracts text across both single- and two-line Pakistani formats using a whole-crop and split scoring approach. ByteTrack extends this logic to video and webcam feeds without redundant OCR runs per frame, while a queuing system decouples frame capture from recognition timing across a shared input interface.

This vision engine feeds a full web application backed by a three-table Neon PostgreSQL schema, featuring a public portal for vehicle registration/status lookups and an administrator portal for request reviews and approvals. Thirteen scripted functional tests validated the application layer's end-to-end behavior and security boundaries. Beyond system implementation, the project serves as a case study in AI-assisted software engineering, illustrating how agentic coding tools accelerated model integration, OCR migration, and concurrency handling while reserving final authority over critical access decisions for the developer.

7.2 Future Enhancements

Several extensions follow naturally from the system's current state rather than requiring architectural change:

- **Recognition accuracy on edge cases:** Expanding the training and test data to cover damaged, angled, or unusually formatted plates would close the gap this report identifies between validated and real-world performance (Section 6.4).
- **Automated reviewer notification:** Alerting an administrator when a new registration request arrives, rather than relying on manual queue checking.
- **Date-range reporting and export:** Using the existing timestamped access-log data to support administrator-facing reports with CSV or PDF export.
- **Physical gate integration:** Connecting the verification pipeline to a live IP/RTSP camera feed and a physical gate controller, moving from the current browser-camera and image-upload inputs to a fully automated entry point.
- **Stronger identity management:** Replacing the current short-lived, in-memory administrator session with a persistent, production-grade identity provider ahead of any real deployment.
- **Scalable image storage:** Moving vehicle images out of the database and into dedicated object storage as the registered-vehicle count grows.

Taken together, SENTRYX's current state is a validated software foundation rather than a finished product: the core recognition and authorization logic are proven, and what remains is primarily deployment hardening rather than unresolved research questions.

REFERENCES

- [1] TransCore, "Parking & Access Control," TransCore. [Online]. Available: <https://transcore.com/rfid/parking-access-control>
- [2] Nedap Identification Systems, "RFID Vehicle Access Control Explained," Nedap Identification Systems. [Online]. Available: <https://www.nedapidentification.com/insights/rfid-vehicle-access-control-explained/>
- [3] Parking BOXX, "Parking Access Control Systems | RFID LPR," Parking BOXX. [Online]. Available: <https://parkingboxx.com/parking-access-control-systems>
- [4] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic License Plate Recognition (ALPR): A State-of-the-Art Review," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 23, no. 2, pp. 311–325, Feb. 2013.
- [5] Lubna, N. Mufti, and S. A. A. Shah, "Automatic Number Plate Recognition: A Detailed Survey of Relevant Algorithms," *Sensors*, vol. 21, no. 9, p. 3028, May 2021.
- [6] A. D. Iriawan and A. Sunyoto, "Automatic License Plate Recognition System in Indonesia Using YOLOv8 and EasyOCR Algorithm," in *Proc. 6th Int. Conf. Information and Communications Technology (ICOIACT)*, 2023, pp. 1–6.
- [7] Ultralytics, "YOLOv8," GitHub documentation, 2023. [Online]. Available: <https://docs.ultralytics.com/models/yolov8>
- [8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Software, Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] M. Al-Batat et al., "Automatic Jordanian License Plate Detection and Recognition System Using Deep Learning Techniques," *Sensors*, vol. 23, no. 12, p. 5567, Jun. 2023.
- [10] "Advanced Deep Learning Techniques for Automated License Plate Recognition," *Sci. Rep.*, vol. 15, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-24967-9>
- [11] N. Karthik, "Overview on PPOCR Architecture," *Medium — IceApple DevLogs*, 2024. [Online]. Available: <https://medium.com/iceapple-tech-talks/paddle-model-architecture-overview-11afa6c79cd1>
- [12] PaddleOCR Team, "PaddleOCR 3.0 Technical Report," *arXiv:2507.05595*, Jul. 2025.
- [13] "A Robust Deep Learning Framework for Bangla License Plate Recognition Using YOLO and Vision-Language OCR," *arXiv:2603.10267*, Mar. 2026.
- [14] Roboflow, "Vehicle Registration Plates Dataset," *Roboflow Universe*, 2024. [Online]. Available: <https://universe.roboflow.com/yolo11-pq5fz/vehicle-registration-plates-trudk-huupd>
- [15] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, X. Yuan, P. Luo, W. Liu, and J. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 1–21.
- [16] GitHub, "GitHub Copilot," *GitHub*, 2023. [Online]. Available: <https://github.com/features/copilot>
- [17] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv preprint arXiv:2302.06590*, 2023.
- [18] Anthropic, "Claude Code," *GitHub Repository*, 2025. [Online]. Available: <https://github.com/anthropics/claude-code>
- [19] L. Theivendra, "Transform monolithic Java applications into microservices with the power of AI," IBM Developer, 2025. [Online]. Available: <https://developer.ibm.com/tutorials/transform-monolithic-java-applications-into-microservices-with-the-power-of-ai/>

- [20] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024, doi: 10.1109/TSE.2023.3334955.
- [21] K. Horikawa et al., "Do Agent-Generated Tests Contribute to Code Coverage? An Empirical Study of Test Generation in Agentic Coding Tools," in *Proc. 23rd Int. Conf. Mining Software Repositories (MSR)*, 2026, arXiv:2603.13724.
- [22] J. Spracklen et al., "We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs," in *Proc. 34th USENIX Secur. Symp. (USENIX Security 25)*, 2025, pp. 1–18.
- [23] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," *Commun. ACM*, vol. 68, no. 2, pp. 96–105, 2025.
- [24] N. Plavac, S. A. Amirshahi, M. Pedersen, and S. Triantaphillidou, "Performance of automatic license plate recognition systems on distorted images," *J. Imaging Sci. Technol.*, vol. 68, no. 6, pp. 060401-1–060401-12, Nov. 2024.
- [25] Shiplight AI, "Agent-native autonomous QA in 2026: Complete guide," *Shiplight AI Technical Reports*, 2026.
- [26] Symphony Solutions, "AI agents in 2026: The future of autonomous software," *Symphony Solutions White Papers*, 2026.